LAB WORK - SOFTWARE DEFINED NETWORKING AND OPENFLOW

1. INTRODUCTION

1.1 MOTIVATION

In this lab work, structured in a tutorial/walkthrough manner, we will dive into the concept of Software-defined networking (SDN). For such journey we will have to explore the OpenFlow protocol and the Mininet network emulator. This lab is divided into two parts: in the first part we will follow the step-by-step guide prepared by the Mininet developers to understand the basics of SDN and OpenFlow, and in the second part we will apply a set of network rules using the SDN paradigm.

SDN is an approach to network management that enables dynamic and programmatically efficient network configuration to improve network performance and monitoring. SDN is meant to improve the static architecture of traditional networks and can be employed to centralize network intelligence in one (or more) network component by disassociating the forwarding process of network packets (data plane) from the routing process (control plane). The control plane consists of one or more controllers where the whole intelligence is incorporated. Its architecture has three layers, the application layer, the control layer and the infrastructure layer.

OpenFlow[1,2] is a communications protocol that allows remote access of a network to a network switch or router's forwarding plane. It was initially defined as the communication protocol in SDN architectures that allowed the SDN controller to communicate directly with the forwarding plane, which consists of network devices such as switches and routers, both physical and virtual (hypervisor-based), so it is more responsive to changing business requirements.

Mininet[3] is a network emulator capable of creating a network of virtual hosts, switches, controllers, and links. It hosts run standard Linux network software, and its switches support OpenFlow for highly flexible custom routing and Software-Defined Networking. Mininet supports research, development, learning, prototyping, testing, debugging, and any other tasks that could benefit from having a complete experimental network on a laptop or other PC.

1.2 LEARNING OUTCOMES

At the end of this lab, you should know:

- The basic concepts about software defined networking;
- To prepare virtualized network environments, through *mininet* environment, to apply and evaluate network rules;

---

[1] McKeown, Nick; et al. (April 2008). "OpenFlow: Enabling innovation in campus networks". ACM SIGCOMM Computer Communication Review. 38 (2): 69–74.

[2] https://opennetworking.org/wp-content/uploads/2014/10/openflow-switch-v1.5.1.pdf

[3] http://mininet.org/

- The purpose and structure of the most important OpenFlow messages;
- How to configure an SDN controller to manage a set of SDN-capable devices.

2. PREPARATION

For this lab we will follow the OpenFlow/Mininet tutorial prepared by the Mininet developers, which can be found here: https://github.com/mininet/openflow-tutorial/wiki This tutorial was developed long time ago, but it has been updated – in some sections - over the years.

It is recommended that you complete the first four steps of the tutorial before the lab class, i.e. up to the *Learn Development* section, inclusive. The following tasks are based on this assumption!

For the setup process you will have to make a choice: use the VM imaged provided in the tutorial, containing the version 2.2 of Mininet and all the remaining tools (recommended), or perform a custom installation using the latest Mininet version from the repositories. If you chose the second option, be aware that some of the instructions in the tutorial might be outdated.

3. TASKS

This lab consists in 3 tasks. Following the tutorial mentioned before, we will start by changing the hub behavior mode of the switch up to a learning switch, i.e., once we know the location of a given node, we send a given packet to there instead of flooding all the ports. Then we will go into a flow-based approach (as SDN should be) where, following the previous rationale, the controller will install flows in the switch, so subsequent packets of a given flow are handled without the intervention of the SDN controller. Finally, we will develop a very simple, and *naïve*, QoS-based SDN solution over a custom topology.

The tutorial webpage (https://github.com/mininet/openflow-tutorial/wiki/Create-a-Learning-Switch) contains a lot of information, although some of them is repeated because several SDN controllers can be used. You can select the SDN controller you want but, given the time available to perform this lab, it is recommended to stick with the POX controller.

3.1 ACTING LIKE A LEARNING SWITCH

In this task you should modify the provided `of_tutorial.py` file, which is responsible for the behavior of the SDN controller, so it can act like a learning switch. Carefully verify the content of the file, where the SDN controller is acting like a hub, and then follow the recommendations on the tutorial webpage to turn it into a learning switch. In this task no flows should be installed in the switch.

Carefully report on everything that you have done to achieve the goal of this task. As important as reporting on the modifications on the controller's code, is the report on the captures made on the several network interfaces, i.e. on the several hosts, the switch ports, and on the communication channel between the switch and the SDN

controller. `tcpdump` should be used to collect such traffic (or to verify the absence of it).

3.2 Flow-based network (SDN approach)

In this task you should modify, again, the `of_tutorial.py` file in order to have a flow-based network. Again, follow the (important) information available on the tutorial website. In the end, the control traffic between the switch and the controller should be considerably reduced when compared to the previous approach.

The recommendations on the report are the same as listed in the previous task.

3.3 Custom topology with Quality-of-Service

In this task we will leverage on the concepts learned before. But first, we will stop working on the topology used so far, created by the Mininet, and we will build our own topology. Mininet allows you to create custom topologies, specifying different characteristics for the different connections, e.g. bandwidth, delay, packet loss, etc.[4]

Consider the network topology of Figure 1, represented by 6 switches and 6 end-nodes. An SDN controller is also available, but it is not represented, as well as its connections with the several switches, following an out-of-band configuration. As you can see, there are connections with more capabilities than others, which means that a smart routing scheme should be provided.

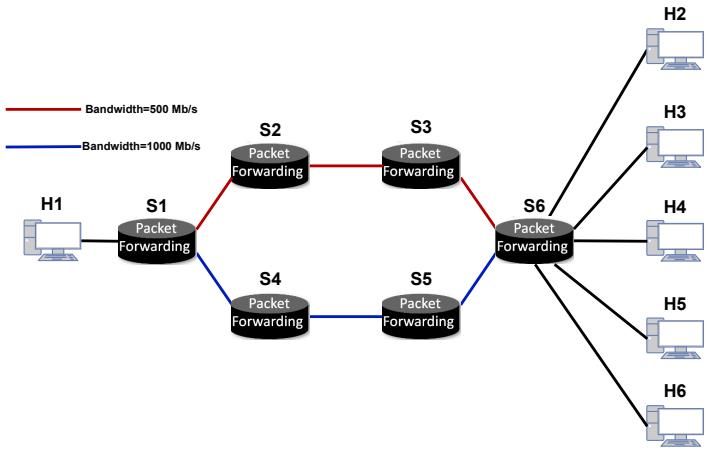


Figure 1 – Custom topology for Task 3.

The goal of this task is to instruct the SDN controller to install the double number of flows in the blue connections when compared to the red connections, i.e. if there are 3 active flows in the network, 2 will go through the blue connections and 1 through the red connections[5]. Do not instruct the SDN with hardcoded rules, such as traffic belonging

[4] http://mininet.org/walkthrough/#custom-topologies

[5] Yes, 1 flow can be more network demanding than 50 000 flows, but in this exercise you will assume that all nodes generate flows with the same characteristics, i.e., equal bitrate for all.

to h1 and h2 goes through blue connections, because the order traffic injection must (and should) be changed over time.

4. DELIVERY

A report (max of 20 plain pages 10 ≤ letter size ≤ 12 and 1.0 ≤ spacing ≤ 1.5 lines) should be delivered in due time. If annexes are required, these must not exceed 5 pages. The report is submitted through Moodle, and is due on: **CHECK DATES ON MOODLE.**

The report should be organized in such a way that includes the following sections: cover page with proper identification, introduction, objectives, experimental methodologies, result analysis and conclusions. **Obtained results must be analysed and explained.**

You must also submit all the files that were changed/modified during this lab.